

Kubernetes and Cloud Native Associate - KCNA

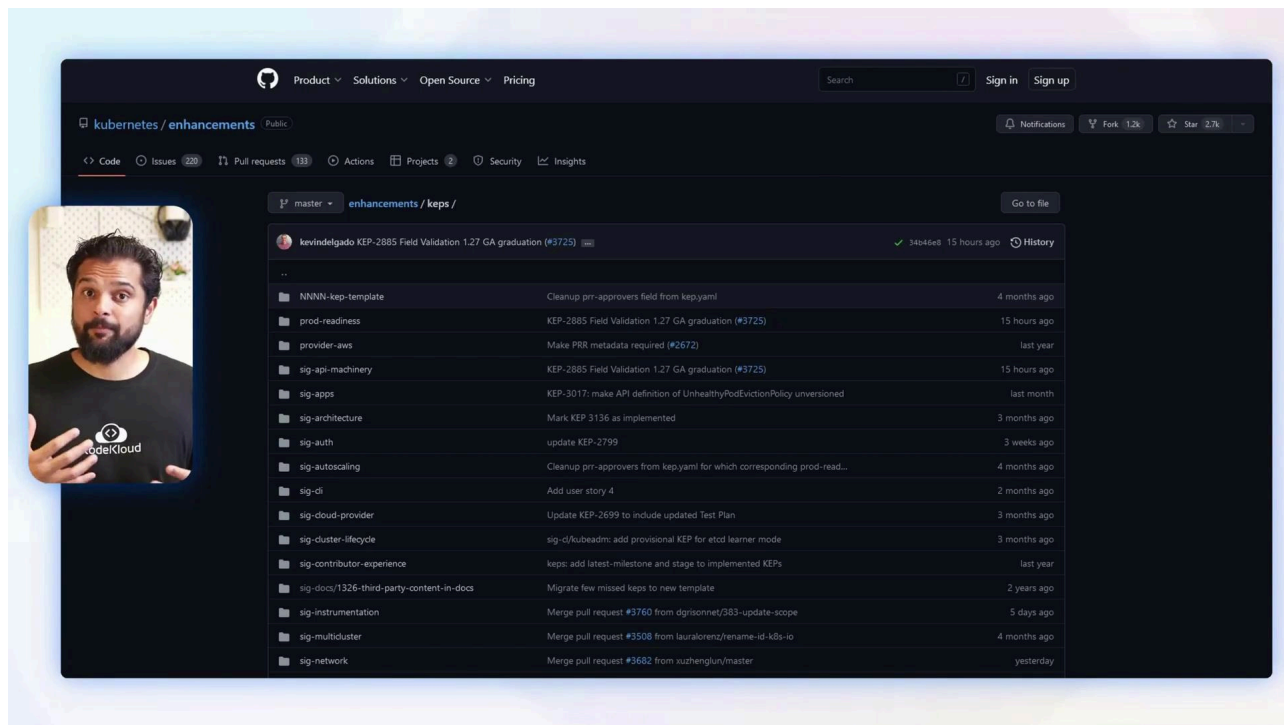
Cloud Native Architecture

Kubernetes Enhancement Proposals strongKEPsstrong

Kubernetes continually evolves with innovative ideas for improvements. To manage these ideas effectively, Kubernetes employs a standardized process known as Kubernetes Enhancement Proposals (KEPs). KEPs offer a structured approach for proposing, discussing, and implementing significant changes through various release stages—alpha, beta, and ultimately GA.

For instance, if you want to understand the reasoning behind a major change—such as the recent update where service tokens are no longer automatically created—reviewing the corresponding KEP will provide the detailed motivation and discussions behind the decision.

If you have a proposal for a new Kubernetes feature, like enhancing usability or bolstering security, you would write a KEP outlining your idea. This process invites feedback and suggestions from the community, ensuring continuous improvement and alignment with industry trends.



The KEP process not only simplifies tracking ongoing development and improvement initiatives but also helps Kubernetes stay abreast of evolving industry standards.

Why Use KEPs Instead of GitHub Issues?

While GitHub issues are excellent for reporting bugs and suggesting features, they lack the structured framework necessary for major changes. KEPs serve several critical roles:

1. They standardize the method for community members to propose and discuss ideas.
2. They facilitate detailed and structured discussions for a thorough evaluation of proposed changes.
3. They provide a formal process for SIGs (Special Interest Groups) to review and either approve or reject proposals, ensuring well-managed enhancements across multiple releases.

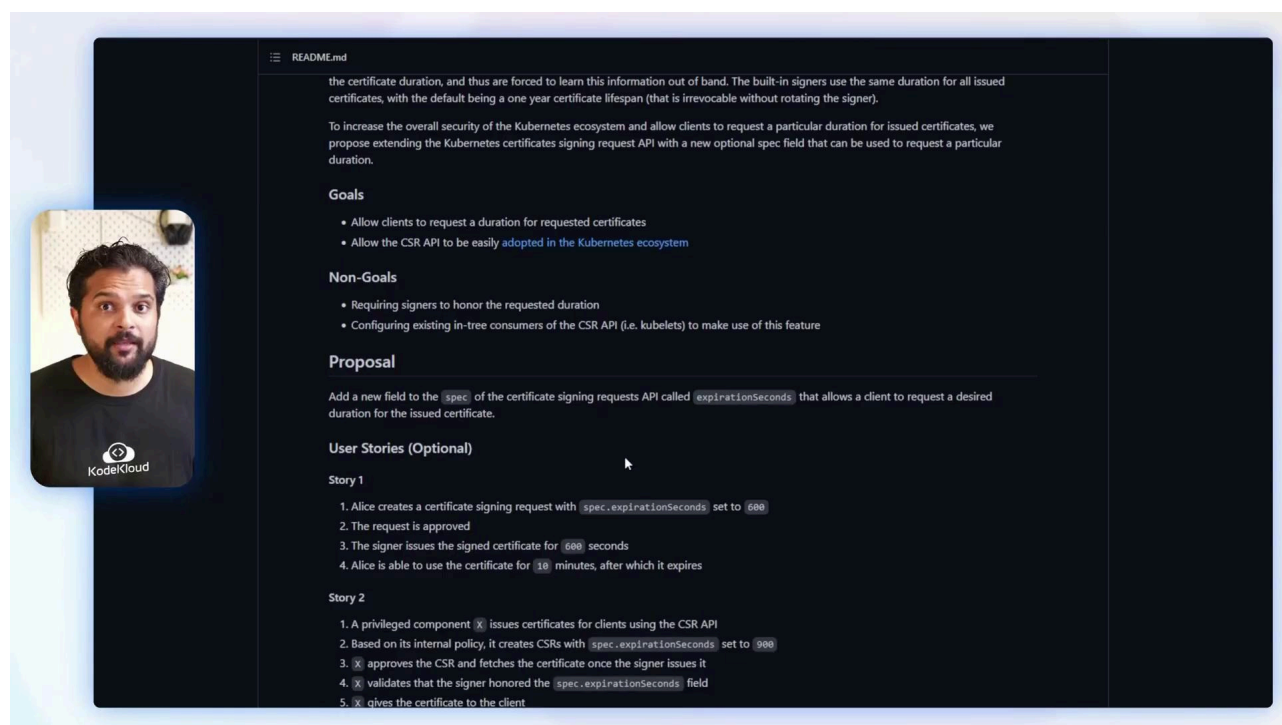
SIGs organize KEPs by areas such as applications, cluster lifecycle, data management, networking, storage, and security. Each SIG votes on proposals, and

approved changes are scheduled for implementation in upcoming Kubernetes releases.

KEP Structure and Content

Every KEP adheres to a standard naming convention with a four-digit number and a descriptive title. Each proposal must include the following elements:

- Summary of the proposal
- Clearly defined goals and non-goals
- The motivation behind the proposal
- A detailed user story
- Risks and mitigation strategies
- Test plans and graduation criteria



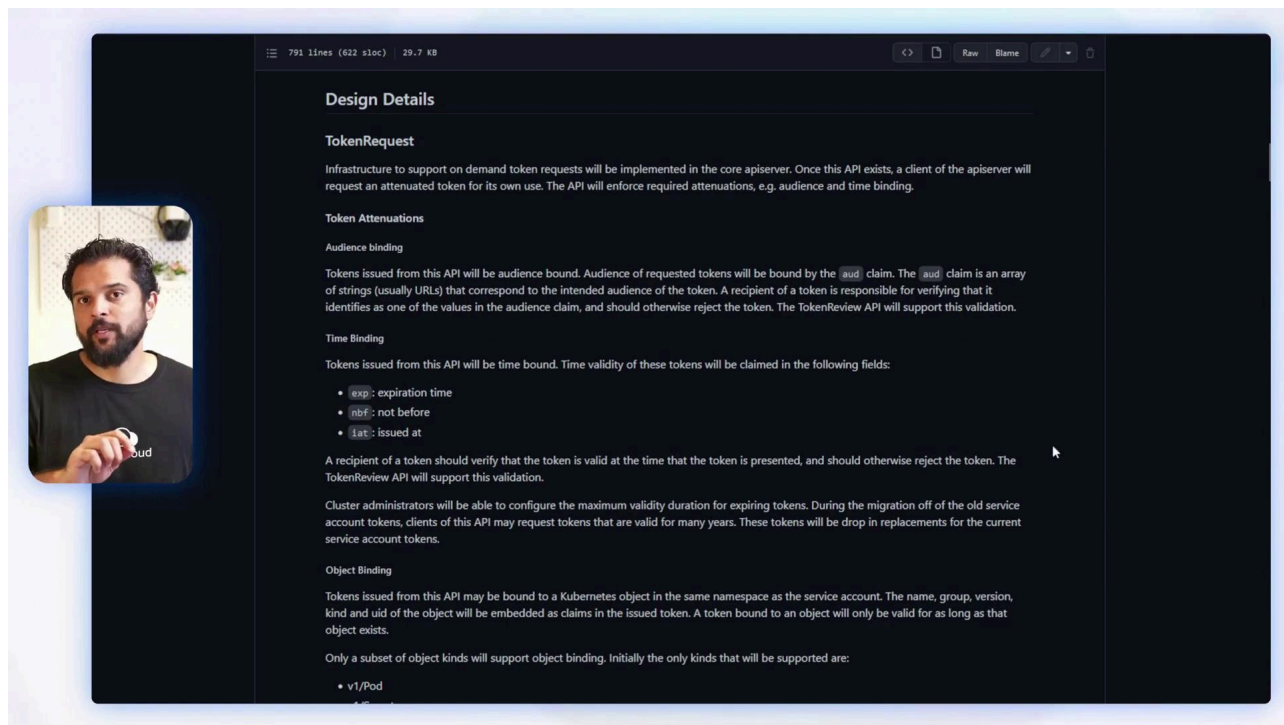
For example, the KEP for the Kubernetes dry run feature—introduced over two years ago—details its benefits. Dry run mode allows you to send requests to endpoints to simulate actions without initiating changes. This proposal includes a test plan, graduation criteria, and usage examples that demonstrate how to invoke it with the

kubectrl command. Below is a sample implementation for parsing a dry run flag in Go:

```
func ParseDryRun(dryRunFlag string) (DryRunEnum, error) {
    if dryRunFlag == "" {
        klog.Warningf("The unset value for --dry-run is deprecated and a value will be required in the future.
        // This warning will eventually become a fatal error.
    }
    b, err := strconv.ParseBool(dryRunFlag)
    if err != nil { // The flag is not a boolean.
        switch dryRunFlag {
            case "client":
                return DryRunClient, nil
            case "server":
                return DryRunServer, nil
            case "none":
                return DryRunNone, nil
            default:
                return DryRunNone, fmt.Errorf("Invalid dry-run value (%v). Must be 'client', 'server', or 'none'")
        }
    }
    if b { // Flag was a boolean and indicates true, run client-side.
        return DryRunClient, nil
    }
    return DryRunNone, nil
}
```

Another recent KEP titled "1205 Bound Service Account Tokens" tackles security concerns related to token provisioning. Traditionally, Kubernetes used JSON Web Tokens (JWTs) that were not audience-specific, potentially leading to security risks if a token was compromised. In such cases, unauthorized access may occur until keys are manually rotated—a process that is often overlooked.

This KEP introduces a token request API that generates tokens on demand, binding them to a specific audience while including an expiry date. The proposal details the implementation strategy, offers example code, and outlines test plans to validate its functionality.



Below is an excerpt of the Go code defining the related structures:

```
type ProjectedVolumeSource struct {
    Sources      []VolumeProjection
    DefaultMode  *int32
}

type VolumeProjection struct {
    Secret                *SecretProjection      `json:"secret,omit`
    DownwardAPI           *DownwardAPIProjection `json:"downwardAPI`
    ConfigMap             *ConfigMapProjection   `json:"configMap,om`
    ServiceAccountToken   *ServiceAccountTokenProjection `json:"serviceAcco`
}

// ServiceAccountTokenProjection represents a projected service account token volu
// This projection can be used to insert a service account token into the pod's ro
// filesystem for use against APIs (Kubernetes API Server or otherwise).
type ServiceAccountTokenProjection struct {
    // Audience is the intended recipient identifier of the token.
    Audience string
    // ExpirationSeconds defines the desired token validity duration.
    // As the token nears expiration, the kubelet volume plugin will rotate it pro
    ExpirationSeconds int64
}
```

```
// Path specifies the relative file path where the token will be projected.  
Path string  
}
```



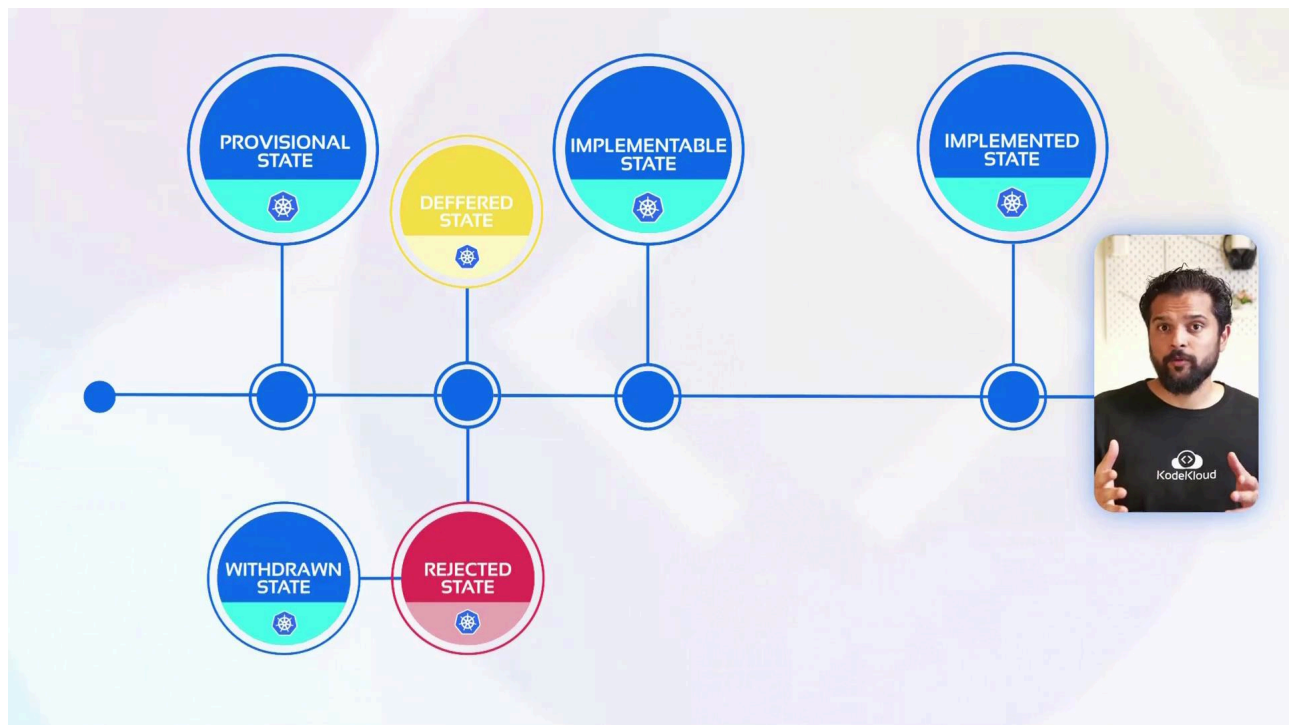
Note

The graduation criteria for each feature are clearly defined in the KEP. For example, the token request API started as an alpha feature in version 1.10, moved to beta in version 1.12, and is planned for GA in version 1.20.

KEP Lifecycle

Each KEP undergoes a precise lifecycle that ensures proposals are thoroughly evaluated and tracked:

- **Provisional:** The proposal is defined and accepted by the SIG.
- **Implementable:** After validation by approvers, the KEP becomes ready for implementation.
- **Implemented:** The changes are fully integrated following successful implementation.
- **Deferred:** The KEP is deferred if no work is actively undertaken.
- **Rejected:** The proposal is rejected if it fails to meet necessary criteria.
- **Withdrawn:** The author withdraws the KEP.
- **Replaced:** The proposal is superseded by a new, updated KEP.



This disciplined lifecycle process ensures every proposal is meticulously considered, implemented, and communicated to the Kubernetes community, contributing significantly to its robustness and continued evolution.

Additional Resources

- [Kubernetes Documentation](#)
- [Kubernetes Basics](#)
- [Docker Hub](#)
- [Terraform Registry](#)



Watch Video

Watch video content

[Previous](#)[◀ Serverless](#)[Next](#)[Kubernetes SIG ▶](#)

Beta

Kubernetes and Cloud Native Associate - KCNA 